ICPC Reference

jeedrubio

May 3, 2025

Contents

Setup

Template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ld = long double;
using vll = vector<ll>;
using ii = pair<ll, ll>;
using vii = vector<ii>;

void solution() {}

int main() {
   ios_base::sync_with_stdio(false);
   cin.tie(nullptr); cout.tie(nullptr);
   ll t = 1;
   // cin >> t;
   while (t--) { solution(); }
   return 0;
}
```

Number Theory

Prime Sieve

```cpp
vll prime_sieve(ll n) {
   vector<bool> sieve(n + 1, true);
   vll res;
   sieve[0] = sieve[1] = false;
   for (ll i = 2; i*i <= n; i++) {
      if (sieve[i]) {
         res.push_back(i);
         for (ll j = i*i; j <= n; j += i) {
            sieve[j] = false;
         }
      }
   }
   return res;
}
```

Prime Factors

```cpp
vll prime_factors(ll n) {
   vll res;
   for (auto p : primes) {
      while (n % p == 0) {
         n /= p;
         res.push_back(p);
      }
   }
   if (n > 1) {
      res.push_back(n);
   }
   return res;
}
```

Find Divisors

```cpp
vll find_divisors(ll n) {
   vll divisors;
   for (ll i = 1; i*i <= n; i++) {
      if (n % i == 0) {
         divisors.push_back(i);
         ll mirr = n / i;
         if (mirr != i) {
            divisors.push_back(mirr);
         }
      }
   }
   return divisors;
}
```

Sum Formulas

Natural numbers from 1 to n

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Even numbers from 2 to 2n

$$\sum_{i=1}^{n} 2i = n(n+1)$$

Odd numbers from 1 to 2n - 1

$$\sum_{i=1}^{n} (2i-1) = n^2$$

Powers from a^0 to a^n

$$\sum_{i=0}^{n} a^i = \frac{1-a^{n+1}}{1-a}$$

Squares from 1 to n^2

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

Graphs

DFS

```
vector<vll> adj; // adjacency list
int n; // number of vertices
vector<bool> visited;

void dfs(ll v) {
   visited[v] = true;
   for (auto u : adj[v]) {
      if (!visited[u])
         dfs(u);
   }
}
```

BFS

```
vector<vll> adj; // adjacency list
ll n; // number of nodes
ll s; // source vertex

queue<ll> q;
vector<bool> used(n);
vll d(n); // path lengths from s to i
vll p(n); // parents

void bfs() {
   q.push(s);
   used[s] = true;
   p[s] = -1;
   while (!q.empty()) {
      ll v = q.front();
      q.pop();
      for (ll u : adj[v]) {
         if (!used[u]) {
            used[u] = true;
            q.push(u);
            d[u] = d[v] + 1;
            p[u] = v;
         }
      }
   }
}

vll get_path(ll u) {
   if (!used[u]) {
      return {};
   }
   vll path;
   for (ll v = u; v != -1; v = p[v])
      path.push_back(v);
   reverse(path.begin(), path.end());
   return path;
}
```

Dijkstra

```
constexpr ll INF = 1000000000;
vector<vii> adj;

void dijkstra(ll s, vll &d, vll &p) {
   ll n = adj.size();
   d.assign(n, INF);
   p.assign(n, -1);
   vector<bool> u(n, false);

   d[s] = 0;
   for (ll i = 0; i < n; i++) {
      ll v = -1;
      for (ll j = 0; j < n; j++) {
         if (!u[j] && (v == -1 || d[j] < d[v]))
            v = j;
      }

      if (d[v] == INF)
         break;

      u[v] = true;
      for (auto edge : adj[v]) {
         ll to = edge.first;
         ll len = edge.second;

         if (d[v] + len < d[to]) {
            d[to] = d[v] + len;
            p[to] = v;
         }
      }
   }
```

```
}

vll get_path(ll s, ll t, vll const& p) {
   vll path;

   for (ll v = t; v != s; v = p[v])
      path.push_back(v);
   path.push_back(s);

   reverse(path.begin(), path.end());
   return path;
}
```

Data Structures

Segment Tree

```
vll arr, segtree;

void build(ll node, ll l, ll r) {
   if (l == r) {
      segtree[node] = arr[l];
      return;
   }
      ll mid = (l + r) / 2;
      build(2 * node, l, mid);
      build(2 * node + 1, mid + 1, r);
      segtree[node] = segtree[2 * node]
         + segtree[2 * node + 1];
}

void update(ll node, ll l, ll r, ll idx, ll val) {
   if (l == r) {
      arr[idx] = val;
      segtree[node] = val;
   } else {
      ll mid = (l + r) / 2;

      if (l <= idx && idx <= mid)
         update(2 * node, l, mid, idx, val);
      else
         update(2 * node + 1, mid + 1, r, idx, val);

      segtree[node] = segtree[2 * node]
         + segtree[2 * node + 1];
   }
}

ll query(ll node, ll tl, ll tr, ll l, ll r) {
   if (r < tl || tr < l)
      return 0;
   if (l <= tl && tr <= r)
      return segtree[node];
   ll tm = (tl + tr) / 2;

   return query(2 * node, tl, tm, l, r)
        + query(2 * node + 1, tm + 1, tr, l, r);
}
```

DP

Top-down

```
ll n;
vector<bool> vis(n, false);
vll dp(n);
vll ve(n);

ll f_dp(ll i) {
   // base cases
   if (i < 0) return INF;
   if (i == 0) return 0;

   if (vis[i]) return dp[i];
   vis[i] = true;
   ll &ans = dp[i];

   // neutral element
   ans = INF;

   // transitions
   ans = min(ans,
      f_dp(i - 1) + abs(ve[i] - ve[i - 1])
   );

   ans = min(ans,
      f_dp(i - 2) + abs(ve[i] - ve[i - 2])
   );

   return ans;
}
```

Bottom-up

```
ll dp_f(ll n, vll const &ve) {
   vll dp(n);

   // base cases
   dp[0] = 0;
   dp[1] = abs(ve[1] - ve[0]);
```

```
for (ll i = 2; i < n; i++) {
   // transitions
   dp[i] = min(
      dp[i - 1] + abs(ve[i] - ve[i - 1]),
      dp[i - 2] + abs(ve[i] - ve[i - 2])
   );
   }
   return dp[n - 1];
}
```